Team2: Asset monitoring and change detection

Implementation structure:
Languages/formats used: python, JavaScript, SQL, JSON

- Backend :

1)Api →
Files :
assets → Contains code for managing and retrieving asset information
scans→ Contains code for starting scans, scheduling scans, and retrieving scan results/status.
Changes→ Contains code for retrieving and displaying detected changes between asset snapshots.
Alerts→ Contains code for creating, retrieving, and managing security alerts generated from detected changes.

2)Services→
files:
asset service→ Contains code for asset-related operations such as adding assets, retrieving assets, updating assets, and deleting assets.
scan service→ Contains code for running scanners, collecting scan results, and storing them.
change service→ Contains code for comparing old and new snapshots, detecting changes, and generating change records.
alert service→ Contains code for creating alerts from detected changes and sending notifications

3)Models →
files :
asset→ Contains code for the Asset table model, which stores monitored assets
snapshot→ Contains code for the Snapshot table model, which stores timestamped asset states collected during scans.
change→ Contains code for the Change table model, which stores detected differences between old and new snapshots.
alert → Contains code for the Alert table model, which stores generated alerts and their severity levels.

4)Database→
files :
connection → Contains code for creating and managing the database connection/session used by the application.

Postgres → Contains PostgreSQL-specific configuration such as database URL, credentials, and database initialization settings

5)Main.py → main.py is the entry point of the FastAPI application. It starts the backend server and registers all API routes.

- Scheduler layer:
  Celery beat:
  Files→
  1)celeryapp → Contains code for creating and configuring the Celery application and connecting it to Redis.
  2)beat schedule → Contains code for defining when periodic tasks should run.
- Task queue layer:
  Redis:
  Files:
  1)redis client → Contains code for connecting the backend to Redis and pushing/pulling tasks from the queue.
- Workers
  Here contains code to receives task from redis → starts workers → start scheduler

- Scanners:
  Files:
  Nmap scanner→ Contains code for network port scanning using Nmap to detect open ports, services, and OS details.
  Httpx scanner → Contains code for HTTP probing using HTTPX to detect web technologies, titles, status codes, and live hosts.
  Dns scanner→ Contains code to perform DNS lookups and extract DNS records of assets

- Asset state builder:
  Files:
  Normalizer → Contains code for converting scanner outputs from different formats into a single standardized asset structure.

Snapshot builder→ Contains code for creating timestamped snapshots of an assets current state and preparing them for storage.

- Storage:
  PostgreSQL
  Files:
  Asset repository → Contains code for save data into PostgreSQL.
  Snapshot repository→ Contains code for save scans into PostgreSQL.
  Change repository→ Contains code for save detected changes into PostgreSQL.

- Change detection
  Deepdiff
  File:
  Deepdiff engine → Contains code for comparing old and new asset snapshots using DeepDiff and detecting changes.
- Alerts
  Email alert
  Files:
  1)Email alert → Contains code for sending email notifications when important changes are detected.
  2)Alert manager→ Contains code for processing detected changes, assigning severity levels, generating alerts, and triggering notifications.

- Observability:
  Logging
  Files:
  1)logger→ Contains code for logging application events, errors, warnings, and scan activities.
  2)scan logs→ Contains the generated log files created by the logger.

- Frontend
  Sources:
  File:
  1)App → it is the entry point of react application and handles the routing between pages

  Pages →
  File:
  2)Dashboard : shows overall statistics

3)Assets: displays monitored assets
4)Changes: displays detected changes
5)Alerts: displays security alerts

Components→
File:
6)AssetTable → shows assets in table format
7)ChangeTable → displays detected changes
8)AlertTable → displays generated alerts

Services →
Here all api calls are written in one file

Packages :
Here stores project dependencies and scripts

Kali Linux environment setup:

1. Update Kali Linux
2. Install Python, pip, and venv
3. Create the project directory asset-monitoring-platform and move into it.
4. Create a Python virtual environment
5. Activate the virtual environment
6. Install PostgreSQL.
7. Start and enable the PostgreSQL service.
8. Create a PostgreSQL database
9. Create a PostgreSQL user and grant database permissions.
10. Install Redis
11. Start and enable the Redis service.
12. Verify Redis is running.
13. Install Nmap and httpx
14. Verify the Nmap and httpx installation.
15. Install required python packages
16. Install all Python dependencies

17. Verify PostgreSQL, Redis, Python, Celery, Nmap, and HTTPX installations.
18. Create the project folder structure including backend, scheduler, workers, scanners, storage, change detection, alerts, observability, frontend.

Conclusion :
The proposed implementation structure provides a scalable and automated Asset Monitoring and Change Detection platform. By combining FastAPI, Celery, Redis, PostgreSQL, DeepDiff, and React, the system can continuously monitor assets, detect changes in near real time, generate alerts, and present results through a centralized dashboard. The modular architecture separates scheduling, scanning, storage, change detection, alerting, and visualization layers, making the platform easier to develop, maintain, and extend. This implementation enables security teams to quickly identify changes in their attack surface and respond to potential risks more effectively.